pip install requests beautifulsoup4

import requests
 from bs4 import BeautifulSoup

HEADERS = {
 "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) "
 "AppleWebKit/537.36 (KHTML, like Gecko) "
 "Chrome/120.0 Safari/537.36",
 "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,/;q=0.8",
 "Accept-Language": "en-US,en;q=0.9",
 }

def fetch_website_contents(url):
 # add scheme if the user forgot it
 if not url.startswith(("http://", "https://")):
 url = "https://" + url

None

```
try:
    response = requests.get(url, headers=HEADERS, timeout=15)
    response.raise_for_status()
except requests.exceptions.RequestException as e:
    return f"Could not fetch the website. Error: {e}"

soup = BeautifulSoup(response.text, "html.parser")
title = soup.title.string if soup.title else "No title found"

for tag in soup(["script", "style", "nav", "footer", "header",
"img", "input"]):
    tag.decompose()

text = soup.get_text(separator="\n", strip=True)
return f"Title: {title}\n\nPage contents:\n{text}"    explain one
by one python code
```

Here's the code explained piece by piece.

1. Setup

```python
# pip install requests beautifulsoup4
import requests
from bs4 import BeautifulSoup
```

- The comment is the install command for the two third-party libraries.
- `requests` makes HTTP calls (like a browser asking for a page).
- `BeautifulSoup` parses HTML so you can search and clean it.

2. HEADERS

```python
HEADERS = {
    "User-Agent": "Mozilla/5.0 ... Chrome/120.0 Safari/537.36",
    "Accept": "text/html,application/xhtml+xml,...",
    "Accept-Language": "en-US,en;q=0.9",
}
```

- A dictionary of HTTP request headers sent with every request.
- **User-Agent** pretends to be a real Chrome browser. Many sites block the default `python-requests` agent.
- **Accept** says what content types you want (HTML preferred).
- **Accept-Language** asks for English content.
- Defined once at module level so it's reused.

3. Function definition and URL fix

```python
def fetch_website_contents(url):
    if not url.startswith(("http://", "https://")):
        url = "https://" + url
```

- `startswith` accepts a tuple, so it checks both schemes in one call.
- If the user types `example.com`, it becomes `https://example.com`. Without a scheme, `requests` raises an error.

4. Making the request

```python
    try:
        response = requests.get(url, headers=HEADERS, timeout=15)
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        return f"Could not fetch the website. Error: {e}"
```

- `requests.get(...)` sends a GET request with your headers.
- `timeout=15` stops it hanging forever; it gives up after 15 seconds.
- `raise_for_status()` raises an exception for 4xx/5xx responses (404, 500, etc.). Without it, error pages would be treated as success.
- `RequestException` is the base class for all `requests` errors (timeout, DNS failure, connection error, bad status), so one `except` catches them all.
- On failure, it returns a friendly message instead of crashing.

5. Parsing the HTML

```python
    soup = BeautifulSoup(response.text, "html.parser")
    title = soup.title.string if soup.title else "No title found"
```

- `response.text` is the raw HTML as a string.
- `"html.parser"` is Python's built-in parser (no extra install).
- `soup.title.string` gets the text inside `<title>`. The `if soup.title else ...` guard avoids an `AttributeError` when the page has no title.

6. Removing noise

```python
    for tag in soup(["script", "style", "nav", "footer", "header",
"img", "input"]):
```

```
        tag.decompose()
```

- `soup([...])` is shorthand for `soup.find_all([...])`, returning all matching tags.
- `decompose()` deletes each tag and its contents from the tree.
- This strips JavaScript, CSS, navigation, headers/footers, images, and form inputs, leaving mainly the real content.

7. Extracting text

```python
    text = soup.get_text(separator="\n", strip=True)
    return f"Title: {title}\n\nPage contents:\n{text}"
```

- `get_text()` pulls all remaining visible text.
- `separator="\n"` puts each text piece on its own line.
- `strip=True` trims whitespace and drops empty strings.
- The f-string returns the title plus the cleaned content.

Flow summary

URL → add scheme → GET request → check errors → parse HTML → remove junk tags → extract text → return.

Interview-ready notes

- This only works for **static HTML**. Pages rendered by JavaScript (React/Angular SPAs) need Selenium or Playwright.
- Scraping should respect `robots.txt` and the site's terms.
- Possible improvements: use a `requests.Session()` for connection reuse, add retries, and limit the output size.

Want me to show a sample run or add features like link extraction?

```
import os
 from openai import OpenAI
 from pathlib import Path
```

```
from dotenv import load_dotenv
from scraper import fetch_website_contents
from groq import Groq
```

Load .env from the same folder as this Python file

```
load_dotenv(Path(file).resolve().parent / ".env")

api_key = os.getenv("GROQ_API_KEY")
 if not api_key:
 raise ValueError(
 "GROQ_API_KEY not found. Check your .env file."
 )

client = Groq(api_key=api_key)

system_prompt = """You analyze the contents of a website and
 give a short, friendly summary. Ignore navigation menus.
 Respond in markdown."""

def summarize(url):
 website = fetch_website_contents(url)
 response = client.chat.completions.create(
 model="openai/gpt-oss-120b",
 messages=[
 {"role":"system", "content": system_prompt},
 {"role":"user", "content": f"Summarize this website:\n\n{website}"},
 ],
 )
 return response.choices[0].message.content explain each line of code in python
```

Here's each part explained in order. This script is the "brain" that uses the scraper from before and sends the page to an LLM for summarizing.

1. Imports

```python
import os
from openai import OpenAI
from pathlib import Path
from dotenv import load_dotenv
from scraper import fetch_website_contents
from groq import Groq
```

- `os`: lets you read environment variables (`os.getenv`).
- `OpenAI`: imported but **never used**. You can safely delete this line.
- `Path`: modern way to handle file paths.
- `load_dotenv`: reads a `.env` file and loads its key=value pairs into environment variables.
- `fetch_website_contents`: your scraper function from `scraper.py` (same folder).
- `Groq`: the client class for Groq's API, which hosts fast LLM inference.

2. Loading the .env file

```python
load_dotenv(Path(__file__).resolve().parent / ".env")
```

- `__file__` is the path of the current script.
- `.resolve()` converts it to an absolute path.
- `.parent` gives the folder containing the script.
- `/ ".env"` appends the filename (`Path` overloads `/` for joining).
- Result: it always finds `.env` next to the script, no matter where you run Python from.

3. Reading the API key

```python
api_key = os.getenv("GROQ_API_KEY")
if not api_key:
    raise ValueError("GROQ_API_KEY not found. Check your .env
file.")
```

- `os.getenv` returns the value, or `None` if missing.
- `if not api_key` catches both `None` and an empty string.

- `raise ValueError` stops the program early with a clear message instead of failing later with a confusing auth error.
- Keeping keys in `.env` (and out of code) avoids leaking secrets to Git.

4. Creating the client

```python
client = Groq(api_key=api_key)
```

- Creates the object used to talk to Groq's API. Created once and reused for every call.

5. System prompt

```python
system_prompt = """You analyze the contents of a website and
give a short, friendly summary. Ignore navigation menus.
Respond in markdown."""
```

- A triple-quoted multi-line string.
- The **system prompt** sets the model's role and rules: summarize, keep it short and friendly, skip menus, use markdown.

6. The function

```python
def summarize(url):
    website = fetch_website_contents(url)
```

- Takes a URL and calls your scraper, which returns `"Title: ...\n\nPage contents: ..."` as one string.

7. Calling the model

```python
    response = client.chat.completions.create(
        model="openai/gpt-oss-120b",
        messages=[
```

```
            {"role":"system", "content": system_prompt},
            {"role":"user",   "content": f"Summarize this
website:\n\n{website}"},
        ],
    )
```

- `client.chat.completions.create(...)` sends a chat request.
- `model`: `openai/gpt-oss-120b` is an open-weight 120B model hosted on Groq.
- `messages` is the conversation as a list of dicts:
    - **system**: instructions (the prompt above).
    - **user**: the actual task, with the scraped text injected via an f-string.

8. Returning the answer

Python
```
    return response.choices[0].message.content
```

- The response contains a list of `choices` (usually one).
- `[0]` takes the first, `.message.content` is the generated text.

Flow summary

Load .env → get key → create client → `summarize(url)` → scrape page → send system + user messages → return model's text.

Interview-ready notes

- **Prompt structure**: system = behavior, user = task/data. This is the standard pattern for chat LLMs.
- **Context limits**: very long pages can exceed the model's token limit. Truncate first, e.g. `website[:20000]`.
- **Error handling**: wrap the API call in `try/except` for rate limits and network errors.
- **Usage example**:

Python
```
print(summarize("example.com"))
```

- Groq's API is OpenAI-compatible, which is why the unused `OpenAI` import could even replace `Groq` by changing `base_url`.

Want me to add truncation and error handling to make it more robust?

Interview-ready notes

- Prompt structure: system = behavior, user = task/data. This is the standard pattern for chat LLMs.
- Context limits: very long pages can exceed the model's token limit. Truncate first, e.g. `website[:20000]`.
- Error handling: wrap the API call in `try/except` for rate limits and network errors.
- Usage example:

It looks like your message got cut off after "Usage example:". I'm guessing you want the usage example (and maybe the other notes) expanded, so here's the full version.

Usage example

```python
if __name__ == "__main__":

    print(summarize("example.com"))
```

Run it from the folder containing the script:

```
python summarizer.py
```

Output is a short markdown summary of the page. The `if __name__ == "__main__":` guard means the code runs only when you execute the file directly, not when another file imports `summarize`.

Making it robust (truncation + error handling)

Python

```
from groq import APIConnectionError, RateLimitError, APIStatusError


MAX_CHARS = 20000


def summarize(url):
    website = fetch_website_contents(url)[:MAX_CHARS]  # avoid
token overflow
    try:
        response = client.chat.completions.create(
            model="openai/gpt-oss-120b",
            messages=[
                {"role": "system", "content": system_prompt},
                {"role": "user", "content": f"Summarize this
website:\n\n{website}"},
            ],
        )
        return response.choices[0].message.content
    except RateLimitError:
        return "Rate limit hit. Please wait and try again."
    except APIConnectionError:
        return "Network problem reaching Groq."
```

```
    except APIStatusError as e:
        return f"API error {e.status_code}: {e.message}"
```

How to say it in an interview

- **Prompt structure**: "I separate the system message, which defines behavior, from the user message, which carries the task and data. It keeps instructions stable while the input changes."
- **Context limits**: "Models have a token limit, so I truncate or chunk long input. For very large pages I'd split into chunks, summarize each, then summarize the summaries (map-reduce)."
- **Error handling**: "External API calls can fail, so I catch rate limits and connection errors separately, and in production I'd add retries with exponential backoff."
- **Security**: "API keys live in `.env`, loaded with `python-dotenv`, never hardcoded or committed."

If your message was meant to say something else, tell me and I'll adjust.